

Vectorized Expectation-Maximization (EM) Algorithm for Gaussian Mixture Models from Scratch

Abstract

This report presents a complete implementation of the Expectation-Maximization (EM) algorithm for Gaussian Mixture Models (GMM) from scratch in R without using any clustering libraries. The implementation includes data generation, E-step for computing posterior probabilities, M-step for updating mixture parameters, and convergence monitoring through log-likelihood computation. The algorithm successfully recovered the underlying cluster structure from synthetic 2D Gaussian data, achieving **100% clustering accuracy** with convergence in just **3 iterations**.

1. Introduction

Gaussian Mixture Models (GMM) are probabilistic models that represent data as a weighted sum of Gaussian distributions. The Expectation-Maximization (EM) algorithm iteratively estimates the parameters when cluster assignments are unknown.

Problem Statement

Given data $X = \{x_1, x_2, \dots, x_n\}$ where each $x_i \in \mathbb{R}^d$, we assume it comes from a mixture of K Gaussian distributions:

$$P(x_i) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}(x_i \mid \mu_k, \Sigma_k)$$

where:

- K is the number of mixture components
- π_k are the mixing coefficients with $\sum_{k=1}^K \pi_k = 1$
- $\mathcal{N}(x_i \mid \mu_k, \Sigma_k)$ is the multivariate Gaussian density
- μ_k and Σ_k are the mean vector and covariance matrix

The goal is to estimate the unknown parameters $\{\pi_k, \mu_k, \Sigma_k\}$ for $k = 1, \dots, K$ without knowing which cluster each point belongs to.

2. Theoretical Background

2.1 Multivariate Gaussian Density

The probability density function of a multivariate Gaussian distribution is:

$$\mathcal{N}(x | \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

where:

- d is the dimensionality
- μ is the mean vector
- Σ is the covariance matrix
- $|\Sigma|$ is the determinant
- Σ^{-1} is the inverse

2.2 Expectation-Maximization Algorithm

The EM algorithm is an iterative method for finding maximum likelihood estimates when the model depends on unobserved latent variables.

E-Step (Expectation)

Compute the posterior probability (responsibility) that data point x_i belongs to component k :

$$\gamma_{ik} = \frac{\pi_k \cdot \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \cdot \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

M-Step (Maximization)

Update the model parameters using the responsibilities:

Effective number of points for component k :

$$N_k = \sum_{i=1}^n \gamma_{ik}$$

Update mixing coefficient:

$$\pi_k = \frac{N_k}{n}$$

Update mean:

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^n \gamma_{ik} x_i$$

Update covariance:

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^n \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^T$$

2.3 Log-Likelihood

The log-likelihood function monitored for convergence:

$$\log L = \sum_{i=1}^n \log \left(\sum_{k=1}^K \pi_k \cdot \mathcal{N}(x_i | \mu_k, \Sigma_k) \right)$$

The EM algorithm guarantees monotonic increase in log-likelihood at each iteration.

3. Methodology

3.1 Data Generation

Synthetic data was generated using two Gaussian clusters with specific parameters to test the algorithm's ability to recover hidden clusters.

Cluster 1 Parameters:

$$\mu_1 = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \Sigma_1 = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$

Cluster 2 Parameters:

$$\mu_2 = \begin{bmatrix} 7 \\ 8 \end{bmatrix}, \Sigma_2 = \begin{bmatrix} 1 & -0.3 \\ -0.3 & 1 \end{bmatrix}$$

Total points: 200 (100 from each cluster)

Data was generated using Cholesky decomposition:

$$X = Z \cdot L + \mu$$

where $Z \sim \mathcal{N}(0, I)$ and L is the Cholesky factor such that $\Sigma = L^T L$.

3.2 Implementation Steps

The EM algorithm was implemented with the following steps:

Step 1: Generate Dataset

- Created 200 synthetic points from 2 Gaussian clusters
- Used Cholesky decomposition for efficient sampling

Step 2: Initialize Parameters

- Set $K=2$ clusters
- Initialized mixing coefficients $\pi = [0.5, 0.5]$
- Initialized means using distance-based method (find two points far apart)
- Initialized covariance matrices with data variance

Step 3: Implemented Gaussian Density Function

- Computed multivariate normal density for each point
- Handled numerical stability issues (singular matrices, overflow)

Step 4: Implemented E-Step

- Computed responsibilities (posterior probabilities) using Bayes' theorem
- Normalized so each row sums to 1

Step 5: Implemented M-Step

- Updated mixing coefficients: $\pi_k = N_k/n$
- Updated means: $\mu_k = (1/N_k) * \sum y_{ik} x_i$
- Updated covariances: $\Sigma_k = (1/N_k) * \sum y_{ik} (x_i - \mu_k)(x_i - \mu_k)^T$

Step 6: Computed Log-Likelihood

- Monitored convergence using log-likelihood
- Stopped when change $< 1e-6$

Step 7: Assigned Final Clusters

- Used hard assignment: $\text{cluster} = \text{argmax}(y_{ik})$

Step 8: Created Visualizations

- Dataset plot
- Final clusters plot with centers
- Log-likelihood convergence plot
- True vs estimated comparison plot

3.3 Numerical Stability

Several measures were implemented for numerical stability:

1. Small diagonal perturbation (1e-6) added to covariance matrices
2. Checks for singular matrices
3. Prevention of overflow in exponential calculations
4. Handling of near-zero probabilities

4. Results

4.1 Algorithm Convergence

The EM algorithm converged in just 3 iterations:

Iteration	Log-Likelihood	Change
1	-671.1794	--
2	-671.0540	0.1255
3	-671.0540	9.98e-08

The monotonic increase in log-likelihood confirms the algorithm's convergence properties.

4.2 Parameter Recovery

The algorithm successfully recovered the true parameters:

Parameter	True Value	Estimated Value	Accuracy
μ_1	(2, 3)	(2.09, 2.95)	98%
μ_2	(7, 8)	(7.12, 7.93)	99%
π_1	0.5	0.4999	100%
π_2	0.5	0.5000	100%

Estimated Covariance Matrices:

Cluster 1:

$$\Sigma_1 = \begin{bmatrix} 0.825 & 0.375 \\ 0.375 & 0.863 \end{bmatrix}$$

Cluster 2:

$$\Sigma_2 = \begin{bmatrix} 0.893 & -0.310 \\ -0.310 & 1.078 \end{bmatrix}$$

4.3 Clustering Performance

- **Clustering Accuracy:** 100%
- **Cluster 1:** 100 points correctly assigned
- **Cluster 2:** 100 points correctly assigned
- **Total Data Points:** 200

4.4 Visualizations Generated

1. **Dataset Plot** (Output/dataset.png): Raw data showing both Gaussian clusters in blue
2. **Final Clusters** (Output/final_clusters.png): Color-coded assignments with star markers for cluster centers

3. **Log-Likelihood** (Output/log_likelihood.png): Convergence monitoring plot showing monotonic increase
 4. **True vs Estimated** (Output/true_vs_estimated.png): Side-by-side comparison of true and estimated clusters
-

5. Learning Outcomes

5.1 Mathematical Understanding

- Multivariate Gaussian distributions and their parameters
- Maximum Likelihood Estimation (MLE) for mixture models
- Expectation-Maximization algorithm theory and proof
- Matrix operations: determinants, inverses, Cholesky decomposition
- Bayes' Theorem for posterior inference

5.2 Programming Skills

- From-scratch implementation without using ML libraries
- Vectorized operations in R
- Numerical stability handling
- Synthetic data generation with known parameters
- Function writing and modular code design
- Error handling and debugging

5.3 Algorithmic Understanding

- E-Step: Computing posterior probabilities using Bayes' theorem
- M-Step: Updating parameters using weighted averages
- Convergence monitoring through log-likelihood
- Soft vs Hard clustering assignments
- Label switching resolution
- Convergence criteria setting

5.4 Key Concepts Mastered

- Gaussian Mixture Models (GMM)
 - EM Algorithm and its convergence properties
 - Maximum Likelihood Estimation (MLE)
 - Bayes' Theorem for posterior inference
 - Multivariate statistics and matrix algebra
 - Algorithmic implementation and debugging
 - Data visualization and interpretation
 - Reproducible research practices
-

6. Conclusion

This project successfully implemented the Expectation-Maximization algorithm for Gaussian Mixture Models from scratch in R. The implementation demonstrated:

1. **Correct parameter estimation** for synthetic Gaussian data
2. **Monotonic convergence** of the log-likelihood
3. **100% accuracy** in cluster assignments
4. **Robust performance** with numerical stability
5. **Effective visualization** of results

The algorithm achieved these key results:

- **Clustering Accuracy:** 100%
- **Iterations to Converge:** 3
- **Final Log-Likelihood:** -671.054
- **Parameter Recovery:** Near-perfect

The project bridges theoretical understanding with practical implementation, providing a solid foundation for more advanced probabilistic modeling and machine learning tasks. The algorithm's ability to recover the true parameters with such high accuracy demonstrates the power of the EM algorithm for unsupervised learning problems.

